

Sistema de Recuperación de Información sobre Tecnología y Software

Alex Moreno Rodriguez and Jery Rodriguez Fernandez

Sistemas de Recuperación de Información — Curso 2025-2026

Abstract. Este informe documenta el diseño, implementación y evaluación de un Sistema de Recuperación de Información (SRI) completo orientado al dominio de tecnología y software. El sistema integra un crawler web bilingüe (inglés/español), un índice invertido basado en Patricia Trie, un índice de co-ocurrencia de términos, una base de datos vectorial ChromaDB, un modelo de recuperación avanzado (GVSM) fusionado con BM25 y búsqueda vectorial, un módulo RAG con el LLM llama-3.1-70b de Cerebras y fallback extractivo, un módulo de búsqueda web automática mediante DuckDuckGo, y una interfaz de usuario React/Vite bilingüe. Se describen la arquitectura modular, las decisiones de diseño, las estadísticas del corpus, el módulo de posicionamiento, la integración end-to-end y un análisis crítico de fortalezas y limitaciones.

1 Introducción

Los sistemas de recuperación de información (SRI) constituyen una disciplina fundamental en ciencias de la computación, cuyo objetivo es encontrar material —generalmente documentos— de naturaleza no estructurada que satisfaga una necesidad de información dentro de grandes colecciones. El presente proyecto implementa un SRI completo orientado al dominio de tecnología y software.

El dominio seleccionado ofrece una riqueza temática diversa: documentación técnica, tutoriales, artículos de ingeniería, publicaciones académicas y blogs especializados, disponibles en inglés y español. Esta diversidad lingüística y temática representa un desafío interesante para los componentes de recuperación y ranking.

El sistema final abarca: adquisición de datos mediante un crawler enfocado, construcción de índices invertidos y de co-ocurrencia, implementación del modelo de recuperación GVSM (Generalized Vector Space Model), base de datos vectorial ChromaDB, mejora del ranking con fusión BM25+GVSM+ChromaDB, módulo RAG con generación aumentada y fallback extractivo, búsqueda web automática con DuckDuckGo, e interfaz visual React/Vite.

2 Dominio Temático

2.1 Justificación

El dominio de tecnología y software fue seleccionado por las siguientes razones:

- Disponibilidad de fuentes de alta calidad y actualizadas en la web (documentación oficial, blogs de ingeniería, repositorios).
- Riqueza terminológica técnica que favorece la evaluación de modelos de recuperación avanzados.
- Cobertura bilingüe natural (inglés/español), lo que permite explorar recuperación multilingüe.
- Relevancia práctica: los usuarios de este tipo de sistemas buscan información técnica con necesidades de información bien definidas.

2.2 Fuentes de Datos

Las semillas de crawling fueron seleccionadas para cubrir diversas categorías dentro del dominio:

- Documentación oficial de lenguajes: Python, Rust, Go, Java, .NET, Kotlin.
- Frameworks y herramientas: React, Angular, Vue, Node.js, Django, Flask.
- Blogs de ingeniería: Netflix Tech Blog, Facebook Engineering, Google Open Source, AWS Architecture, Stack Overflow Blog.
- Recursos académicos y estándares: arXiv (CS), W3C, RFC Editor, Semantic Scholar.
- Referentes técnicos: Martin Fowler, CSS-Tricks, Smashing Magazine.

3 Módulo de Adquisición de Datos

3.1 Arquitectura del Crawler

El módulo de adquisición (`tech_crawler.py`) implementa un crawler web enfocado en tecnología, diseñado con una arquitectura modular organizada en los siguientes componentes reutilizables bajo el paquete `crawler_engine`:

- `SimpleHTMLExtractor`: parser HTML que extrae título, texto visible, enlaces salientes y metadatos de idioma.
- `score_relevance`: función de puntuación basada en presencia de palabras clave técnicas bilingües.
- `canonicalize_url`: normalización de URLs para deduplicación (eliminación de query strings, fragmentos y barras finales).
- Gestión de robots.txt mediante `RobotFileParser` con caché por dominio.

3.2 Proceso de Crawling

El crawler opera con una frontera de prioridad (min-heap) donde cada elemento contiene: prioridad (negativo del puntaje de relevancia), profundidad actual y URL. El proceso sigue los siguientes pasos por cada URL extraída de la frontera:

- Verificación de límites: URLs ya visitadas, límite por dominio, profundidad máxima.

- Validación de robots.txt: se respeta la directiva `Disallow` para el agente `IRS-TechCrawler/1.1`.
- Throttling por dominio: espera configurable (default 1 s) entre accesos al mismo dominio.
- Extracción de contenido: título, texto plano, enlaces salientes e idioma detectado.
- Filtrado de relevancia: documentos con menos de 300 caracteres o sin palabras clave técnicas son descartados.
- Deduplicación: hash SHA-256 del texto para evitar contenido duplicado.
- Almacenamiento en JSONL con campos: `doc_id`, `url`, `title`, `domain`, `crawl_date`, `text`, `language`, `source_type`.

3.3 Detección de Idioma

La detección de idioma opera en cascada con cuatro niveles de confianza decreciente:

1. Metaetiquetas HTML: atributo `lang` del elemento `<html>`, meta `http-equiv='content-language'`, `og:locale`.
2. Cabecera HTTP `Content-Language` retornada por el servidor.
3. Patrones en la URL: subdominios como `es.site.com` o segmentos de ruta `/en/` o `/es/`.
4. Análisis heurístico de stopwords: comparación de vocabulario con listas de stopwords en inglés y español.

3.4 Políticas de Crawling

El crawler implementa las siguientes políticas de uso responsable:

- Respeto a robots.txt con caché de parsers por dominio raíz.
- User-Agent identificado: `IRS-TechCrawler/1.1 (+academic-project)`.
- Delay configurable entre accesos al mismo dominio (default: 1.0 s).
- Límite de páginas por dominio (default: 120) para evitar sobrecarga.
- Extensiones binarias descartadas sin descarga (`.pdf`, `.zip`, imágenes, videos, etc.).

4 Módulo de Indexación

4.1 Preprocesamiento de Texto

El preprocesamiento de texto se implementa en la clase `Index` (`indexer.py`) utilizando spaCy con el modelo `en_core_web_sm`. El pipeline de tokenización aplica los siguientes pasos en orden:

- Lematización mediante el modelo spaCy seleccionado.
- Conversión a minúsculas.
- Normalización Unicode NFKD con eliminación de diacríticos.

- Filtrado de tokens de puntuación, espacios en blanco y marcadores especiales (-PRON-).
- Eliminación de tokens con longitud menor o igual a 2 caracteres.
- Eliminación de tokens no alfabéticos.
- Remoción de stopwords mediante el flag `is_stop` de spaCy y listas personalizadas en inglés y español.

4.2 Índice Invertido: Patricia Trie

El índice invertido se implementa como un Patricia Trie comprimido (clase `PatriciaTrie` en `indexer.py`). Esta estructura almacena términos como caminos comprimidos desde la raíz, donde cada arco representa un prefijo (no un único carácter), reduciendo significativamente el uso de memoria frente a un trie estándar.

Las operaciones implementadas son:

- `insert(word, doc_id)`: inserción con compresión de arcos compartidos y actualización de *posting lists*.
- `search(word)`: búsqueda exacta con retorno de lista de `doc_ids`.
- `get_parcial_AND(terms, min_match, max_candidates)`: recuperación parcial con mínimo de términos coincidentes, ordenada por frecuencia de matches.
- Persistencia JSON con formato flat de nodos indexados (índice en lugar de referencias de puntero).
- Carga incremental: al iniciar, sincroniza automáticamente con los últimos documentos del JSONL sin reconstruir todo el índice.

4.3 Índice de Co-ocurrencia

La clase `CoOccurrenceIndex` (`engine.py`) mantiene estadísticas de frecuencia documental (df) y co-ocurrencia de pares de términos en formato de matriz dispersa CSR (`scipy.sparse.csr_matrix`). Estas estadísticas son utilizadas por el GVSM para calcular correlaciones término-término.

El índice de co-ocurrencia opera con las siguientes características:

- Almacena únicamente el triángulo superior de la matriz de co-ocurrencia para eficiencia.
- Umbral mínimo de co-ocurrencia configurable (default: 3) para filtrar pares espurios.
- Serialización a JSON con formato sparse (arrays de filas, columnas y datos).
- Reconstrucción automática si el número de documentos cambia respecto al caché.

5 Módulo Recuperador: GVSM y Fusiones

5.1 Selección y Justificación del Modelo GVSM

Se implementó el Modelo Vectorial Generalizado (Generalized Vector Space Model, GVSM), propuesto por Wong, Ziarko y Wong (1985) [1]. La selección de este modelo se justifica por:

- El dominio tecnológico presenta alta correlación entre términos técnicos (p.ej., “python” y “django”, “kubernetes” y “docker”), que el GVSM captura mediante la matriz de correlación término-término.
- Supera las limitaciones del VSM clásico al permitir que términos no presentes en la consulta contribuyan a la puntuación si están correlacionados con términos de la consulta.
- La base matemática (álgebra lineal dispersa) es eficiente con `scipy.sparse`, viable para el tamaño del corpus.

5.2 Definición Formal del GVSM

La similitud entre una consulta q y un documento d se calcula como:

$$\text{sim}(q, d) = \frac{\sum_i \sum_j w_i^q w_j^d \text{corr}(t_i, t_j)}{\|q\| \|d\|}$$

Donde w_i^q y w_j^d son los pesos TF-IDF normalizados de los términos t_i y t_j en la consulta y el documento, respectivamente; $\text{corr}(t_i, t_j)$ es la correlación entre términos, calculada mediante similitud coseno sobre los vectores de co-ocurrencia; y $\|q\|$ y $\|d\|$ son las normas euclidianas de los vectores de peso.

5.3 Implementación del GVSM

La implementación en la clase `GeneralizedVectorSpaceModel` (`engine.py`) incluye:

- Cómputo de IDF suavizado: $\log((N + 1)/(df + 1)) + 1$, donde N es el total de documentos y df la frecuencia documental.
- TF normalizado por el máximo TF en el documento/consulta.
- Matriz de correlación término-término dispersa y simétrica a partir del índice de co-ocurrencia.
- Similitud GVSM con operaciones matriciales dispersas de SciPy: $q^T \cdot A_{sub} \cdot d$.
- Caché de la matriz de correlación con invalidación cuando cambia el vocabulario o el umbral `min_cooc`.
- Pre-cómputo de vectores de documentos al inicializar el motor para reducir latencia.

5.4 Fusión Léxico-Semántica con BM25 y ChromaDB

El recuperador fue refinado incorporando BM25 como señal léxica complementaria. La motivación: GVSM puede asignar puntuaciones altas a documentos semánticamente relacionados pero sin los términos exactos de la consulta, mientras que BM25 prioriza la coincidencia exacta con ajuste de longitud.

El esquema de fusión normaliza ambas puntuaciones al rango $[0, 1]$ antes de combinarlas:

$$\begin{aligned}\text{sem_norm}(d) &= \text{GVSM}(d) / \max_d(\text{GVSM}) \\ \text{lex_norm}(d) &= \text{BM25}(d) / \max_d(\text{BM25}) \\ \text{base}(d) &= \alpha \cdot \text{sem_norm}(d) + (1 - \alpha) \cdot \text{lex_norm}(d), \quad \alpha = 0.6 \\ \text{score}(d) &= (1 - \lambda) \cdot \text{base}(d) + \lambda \cdot \text{chroma_norm}(d), \quad \lambda = 0.75\end{aligned}$$

El BM25 implementado incorpora un boost de 2x para coincidencias en el título: la frecuencia efectiva se calcula como $f_{eff} = tf_{cuerpo} + title_weight \cdot tf_{titulo}$, con $title_weight=2.0$.

El conjunto de candidatos para el ranking se forma como la unión de dos fuentes:

- Patricia Trie (`get_parcial_AND`): candidatos que coinciden con al menos `min_match` términos de la consulta.
- ChromaDB (búsqueda vectorial): top- k documentos por similitud coseno en el espacio de embeddings.

Para consultas de un solo término, los candidatos de ChromaDB se filtran adicionalmente mediante coincidencia léxica en título, texto o URL.

6 Base de Datos Vectorial

6.1 Tecnología Seleccionada: ChromaDB

La base de datos vectorial se implementa utilizando ChromaDB (`chroma_store.py`), una base de datos vectorial de código abierto que permite almacenamiento persistente y búsqueda por similitud coseno mediante índices HNSW (Hierarchical Navigable Small World).

6.2 Generación de Embeddings

En la versión actual, los embeddings se generan mediante una función de hashing determinista (`ChromaVectorStore._embed_text`) que mapea tokens a posiciones en un vector de dimensión 384 usando SHA-1. Cada token contribuye con ± 1 en una posición determinada por su hash, y el vector resultante se normaliza a norma unitaria. Este enfoque garantiza reproducibilidad sin dependencias externas de modelos de lenguaje, con la posibilidad de reemplazarse por embeddings semánticos en versiones futuras.

6.3 Integración con el Motor de Búsqueda

Los documentos del corpus se integran en ChromaDB al inicializar el motor. Durante la búsqueda, se ejecuta una consulta vectorial en paralelo con la recuperación por Trie, y los resultados de ambas fuentes se fusionan antes del ranking final.

7 Módulo RAG (Retrieval-Augmented Generation)

7.1 Arquitectura General

El módulo RAG (`rag_engine/`) implementa un pipeline de generación aumentada que integra el recuperador GVSM con el LLM llama-3.1-70b de Cerebras. La arquitectura es modular con separación clara de responsabilidades:

- **RAGConfig**: configuración centralizada cargada desde variables de entorno (`.env`).
- **CerebrasClient**: cliente API con retry exponencial (hasta 3 intentos, backoff 2^n segundos).
- **CerebrasRAGGenerator**: orquestador principal que coordina generación LLM y fallback.
- **ImprovedRAGGenerator**: generador extractivo de respaldo con scoring de sentencias.
- **RAGPipeline**: interfaz pública que mantiene compatibilidad con `main.py`.

7.2 Proceso de Generación

1. Recuperación: el pipeline llama a `retriever.search(query, top_k=5)` para obtener los documentos más relevantes.
2. Construcción del prompt: se construye un prompt estructurado con la consulta y los fragmentos de los documentos recuperados (presupuesto de 2400 caracteres total).
3. Generación LLM: `CerebrasClient` envía el prompt a la API de Cerebras (temperatura 0.7, máximo 1200 tokens).
4. Análisis de suficiencia: el modelo indica en la primera línea de respuesta si los documentos recuperados son suficientes (campo JSON `{"sufficient": bool}`).
5. Truncado: `truncate_answer()` respeta límites de oraciones.
6. Atribución: `extract_used_doc_ids()` detecta qué documentos fueron citados implícitamente mediante overlap de palabras clave de títulos.

7.3 Prompt Engineering

El prompt está diseñado en español para reducir mezcla de idiomas en las instrucciones, con directivas explícitas:

- Responder únicamente con la información presente en los documentos recuperados.
- Indicar explícitamente si los documentos no son suficientes.
- No inventar datos ni rellenar con contenido no respaldado.
- Responder en el mismo idioma de la consulta (inglés o español).
- Mantener la respuesta entre 300 y 1200 caracteres.
- Emitir en la primera línea un objeto JSON con la clave `'sufficient'` (booleano).

7.4 Generador Extractivo de Fallback

Si el LLM de Cerebras no está disponible o falla tras 3 reintentos, el sistema activa automáticamente **ImprovedRAGGenerator**, un generador extractivo que:

- Divide cada documento recuperado en oraciones mediante expresiones regulares.
- Puntúa cada oración por overlap entre sus tokens y los términos de la consulta, ponderado por el score de relevancia del documento.
- Selecciona las mejores oraciones respetando diversidad de fuentes (máximo 2 oraciones por documento) y presupuesto de caracteres.
- Aplica un fallback de último recurso: fragmento del primer documento si no se extraen oraciones relevantes.

7.5 Búsqueda Web como Fallback del RAG

Cuando el LLM indica que los documentos recuperados son insuficientes (`sufficient=false`), el **RAGPipeline** activa el **WebSearchPipeline** para obtener documentos adicionales de la web, los ingesta en el motor mediante `ingest_records()`, y reejecuta la generación con el corpus ampliado.

8 Módulo de Búsqueda Web

8.1 Diseño y Activación Automática

El módulo de búsqueda web (`web_search.py`, `web_search_engine/`) implementa una búsqueda en DuckDuckGo como mecanismo de ampliación del corpus local. La activación es automática y se basa en dos umbrales configurables que deben cumplirse simultáneamente:

- Número de documentos locales recuperados $< \text{min_local_docs}$ (default: 5).
- Puntuación promedio de los resultados locales $< \text{min_score_threshold}$ (default: 0.3).

La combinación de ambos criterios evita activaciones innecesarias.

8.2 Pipeline de Búsqueda

- Consulta a DuckDuckGo HTML: se construye una URL de búsqueda y se parsea el HTML de resultados para extraer URLs mediante expresiones regulares.
- Descarga paralela: hasta 5 URLs se descargan concurrentemente con **ThreadPoolExecutor**, timeout de 12 segundos por URL.
- Extracción de contenido: **SimpleHTMLExtractor** extrae título, texto visible e idioma.
- Persistencia: los registros se guardan en el JSONL principal para uso futuro, evitando repetir búsquedas web para las mismas consultas.

8.3 Integración con el Motor

Los documentos obtenidos de la búsqueda web se integran al motor mediante `GVSMSearchEngine.ingest_records()`, que actualiza: `self.records`, `self.doc_tokens_by_id`, `self.doc_vectors` (vectores GVSM), las estructuras BM25 (`term_doc_freq`, `doc_tf_body`, `doc_len`), y upserta en ChromaDB. Tras la ingesta, el IDF y el índice BM25 se recomputan.

8.4 Comparativa de Módulos

Aspecto	Búsqueda Local (GVSM)	Búsqueda Web (DuckDuckGo)
Fuente	Corpus crawleado	DuckDuckGo (web)
Latencia	< 1 s	5–15 s
Activación	Siempre	Automática (umbral)
Persistencia	JSONL indexado	Se agrega al JSONL
Cobertura	Dominio acotado	Web abierta
Control de calidad	Alto (filtros crawler)	Moderado (extracción HTML)

9 Módulo de Interfaz Visual

9.1 Diseño General

La interfaz de usuario se implementó como una Single Page Application (SPA) utilizando React 18 y Vite como bundler. La aplicación se comunica con el servidor Flask mediante llamadas HTTP asíncronas a los endpoints `/search` y `/rag`.

El diseño visual sigue una paleta de colores neutra (azul pizarra #406d8f como acento, fondos grises suaves) que favorece la legibilidad de contenido técnico. La tipografía usa Nunito Sans.

9.2 Componentes Principales

- **TopNav**: barra de navegación con selector de idioma (ES/EN) y botón de configuración. El cambio de idioma es persistente via `localStorage`.
- **SearchPortal**: componente central que integra el buscador, los filtros por tipo, el panel RAG y la lista de resultados.
- **FilterPill**: botones de filtro con estado `aria-pressed` para accesibilidad.
- `icons.jsx`: iconografía SVG propia.

9.3 Flujo de Interacción del Usuario

1. El usuario escribe una consulta y presiona BUSCAR o Enter.
2. La interfaz lanza dos peticiones paralelas: `/search` (resultados GVSM) y `/rag` (respuesta generativa).
3. Mientras se cargan los resultados, se muestra un estado de carga.

4. El panel RAG aparece primero con indicador de carga.
5. Los resultados se muestran paginados (6 por página) con título clicable, snippet, dominio, fecha y tipo de archivo.
6. Los filtros por tipo permiten refinar los resultados mostrados (aunque en la versión actual son solo visuales).
7. El subtítulo animado rota cada 3 segundos entre frases descriptivas.

9.4 Internacionalización

La interfaz soporta inglés y español de forma completa mediante el módulo `i18n.js`. El idioma seleccionado se persiste en `localStorage` bajo la clave `'sri-language'` y se aplica al atributo `lang` del elemento `<html>`.

9.5 Guía de Uso

1. Asegurar que el servidor Flask esté corriendo: `python3 scripts/main.py serve` (puerto 5000).
2. Instalar dependencias del frontend: `cd webapp && npm install`.
3. Iniciar el servidor de desarrollo: `npm run dev`.
4. Abrir `http://localhost:5173`.
5. Escribir una consulta técnica (ej.: “python machine learning”, “kubernetes deployment”, “React hooks”).
6. Revisar el panel RAG y la lista de resultados.
7. Usar los filtros y la paginación; cambiar idioma desde la barra superior.

10 Módulo de Posicionamiento

10.1 Estrategia de Posicionamiento

El módulo de posicionamiento determina el orden de presentación de los resultados combinando múltiples señales de relevancia. Adopta un enfoque híbrido que prioriza relevancia semántica sobre popularidad.

10.2 Factores de Ranking

- Puntuación de fusión GVSM+BM25+ChromaDB (factor principal, rango $[0, 1]$): combina relevancia semántica, léxica y vectorial.
- Boost por coincidencia de subcadena en título o texto: documentos que contienen literalmente la consulta reciben +0.15 si no están ya en el top- k .
- Rank de fuente como desempate: ante igual puntuación, se prioriza el orden de retorno del motor.
- Orden alfabético de título como último desempate para garantizar estabilidad.

10.3 Posicionamiento Visual

- El panel RAG ocupa la posición más prominente (parte superior).
- Cada resultado muestra título (enlace clicable), snippet (primeros 320 caracteres), dominio, fecha de crawl y tipo de archivo.
- Paginación: 6 resultados por página.
- Etiqueta de tipo (PDF, Imagen, Video, Documento, Otro) en el extremo derecho.
- Las fuentes usadas en la respuesta RAG aparecen como pills clicables bajo el texto generativo.

10.4 Factores Adicionales (No implementados en esta versión)

- Frescura: la fecha de crawl se muestra pero no afecta el ranking.
- Popularidad: no se implementó PageRank explícito.
- Fuente web vs. local: los resultados de búsqueda web se integran sin distinción.

11 Integración Completa del Sistema

11.1 Arquitectura General

El sistema sigue una arquitectura cliente-servidor con tres capas:

- Capa de datos: corpus JSONL + índices persistidos (Patricia Trie JSON, co-ocurrencia JSON, ChromaDB).
- Capa de lógica: servidor Flask (`main.py`) que orquesta `GVSMSearchEngine`, `RAGPipeline` y `WebSearchPipeline`.
- Capa de presentación: SPA React/Vite que consume la API REST.

11.2 Flujo End-to-End

1. Usuario escribe consulta en la interfaz React.
2. Frontend lanza `GET /search?query=...&top_k=80` y `GET /rag?query=...&top_k=5` en paralelo.
3. Servidor Flask tokeniza la consulta y ejecuta recuperación híbrida (Trie + ChromaDB).
4. Ranking por fusión BM25+GVSM+ChromaDB con boost de subcadena.
5. Evaluación de umbral web: si `local_count < 5` y `avg_score < 0.3`, se activa `WebSearchPipeline`.
6. Respuesta `/search`: JSON con lista de documentos ordenados.
7. En paralelo: `RAGPipeline` recupera top-5, construye prompt, llama a Cerebras LLM.
8. Si LLM indica `sufficient=false`: `WebSearchPipeline` amplía corpus y regenera.
9. Respuesta `/rag`: JSON con `answer`, `sources` y `contexts`.
10. Frontend renderiza panel RAG y lista paginada de resultados.

11.3 Manejo de Errores

- Flask CORS: cabecera `Access-Control-Allow-Origin: *`.
- Retry exponencial en Cerebras: hasta 3 intentos (1 s, 2 s, 4 s).
- Fallback extractivo si Cerebras falla.
- Validación de `doc_id`: registros inválidos se ignoran.
- `Promise.allSettled` en frontend: errores en `/search` o `/rag` no bloquean la otra petición.
- ChromaDB opcional: el motor funciona solo con Trie+GVSM+BM25 si no está instalado.

12 Análisis Crítico del Sistema

12.1 Bondades

- Arquitectura modular: separación en `crawler_engine`, `rag_engine`, `web_search_engine` y `engine.py`.
- Bilingüismo nativo: manejo de inglés y español, detección de idioma en cascada.
- Fusión de señales heterogéneas: GVSM + BM25 + ChromaDB produce ranking robusto.
- Resiliencia del RAG: fallback extractivo garantiza disponibilidad.
- Búsqueda web automática: activación transparente por umbral.
- Interfaz accesible: atributos ARIA y soporte bilingüe.
- Carga incremental del Trie: evita reconstruir todo el índice.

12.2 Insuficiencias y Limitaciones

- Embeddings hash: no capturan semántica real; deberían reemplazarse por embeddings de modelos preentrenados.
- Índice Trie no actualizado en tiempo real: los documentos web solo son recuperables vía ChromaDB o BM25, no por búsqueda Trie.
- Escalabilidad limitada: pre-cómputo de vectores y reconstrucción completa de BM25 al ingestar son ineficientes para grandes corpus ($> 100k$ documentos).
- Sin evaluación formal: falta un conjunto de consultas con relevancia anotada (no se pueden medir precisión, recall, NDCG, MRR).
- Filtros de tipo en la UI no funcionales: los `FilterPill` actualizan estado pero no filtran realmente.
- Detección de idioma de consulta: el sistema responde en el idioma de los documentos, no necesariamente en el de la consulta.
- Calidad del scraping web: el extractor HTML simple falla en páginas con renderizado JavaScript.

12.3 Posibles Soluciones a las Deficiencias

- Embeddings semánticos: integrar `sentence-transformers` (ej. `all-MiniLM-L6-v2`).
- Trie dinámico: modificar `ingest_records()` para llamar a `PatriciaTrie.insert()`.
- Índice BM25 incremental: mantener estructuras delta separadas.
- Conjunto de evaluación: construir manualmente 50-100 consultas técnicas anotadas e implementar MAP, NDCG@10, MRR.
- Filtros funcionales: pasar `activeFilters` al endpoint `/search` y filtrar por `file_type` en el servidor.
- Detección de idioma de consulta: aplicar `infer_language_from_text()` a la consulta.
- Scraping con Playwright/Selenium para páginas JS.

13 Manual de Usuario

13.1 Requisitos del Sistema

- Python 3.8+ con dependencias de `requirements.txt`.
- Node.js ≥ 16 y npm.
- Conexión a internet para búsqueda web y API de Cerebras.
- (Opcional) Docker.

13.2 Pasos de Instalación

1. Clonar el repositorio: `git clone <url>`.
2. Crear entorno virtual: `python3 -m venv .venv && source .venv/bin/activate`.
3. Instalar dependencias Python: `pip install -r requirements.txt`.
4. Instalar dependencias del frontend: `cd webapp && npm install`.
5. Configurar clave de Cerebras en `.env`: `CEREBRAS_API_KEY=csk-.....`

13.3 Ejecución del Sistema

1. (Opcional) Crawling: `python3 scripts/tech_crawler.py -max-pages 500 -max-depth 2`.
2. Iniciar servidor backend: `python3 scripts/main.py serve`.
3. Iniciar frontend: `cd webapp && npm run dev`.
4. Abrir `http://localhost:5173`.

13.4 Despliegue con Docker

Pendiente de completar con `Dockerfile` y `docker-compose.yml`. Comando tentativo: `docker compose up -build`.

14 Decisiones de Diseño del Posicionamiento

Las decisiones de diseño se tomaron con los siguientes criterios:

- Relevancia sobre popularidad: en un dominio técnico, la pertinencia del contenido es más valiosa que métricas de popularidad no disponibles.
- Transparencia del ranking: la puntuación `score` se expone en la respuesta JSON.
- Panel RAG en posición prominente: los usuarios con necesidades informacionales directas se benefician más de una respuesta sintetizada.
- Snippet contextual: primeros 320 caracteres, suficientes para evaluar relevancia.
- Etiquetado de tipo: permite filtrar mentalmente antes de hacer clic.

15 Conclusiones Finales

El sistema implementado cumple con todos los requisitos obligatorios: módulo de adquisición (crawler bilingüe con robots.txt), módulo de indexación (Patricia Trie + co-ocurrencia), módulo recuperador no básico (GVSM), base de datos vectorial (ChromaDB), módulo RAG (Cerebras LLM + fallback extractivo), módulo de búsqueda web (DuckDuckGo con activación automática), módulo de posicionamiento (fusión BM25+GVSM+vectorial) e interfaz visual (React/Vite bilingüe).

El proyecto representa una implementación coherente e integrada de los conceptos del curso. Las limitaciones identificadas (embeddings hash, Trie no actualizado con web, ausencia de evaluación formal) constituyen líneas claras de trabajo futuro.

References

1. Wong, S.K.M., Ziarko, W., Wong, P.C.N.: Generalized vector space model in information retrieval. In: Proceedings of the 8th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 18–25. ACM, New York (1985).
2. Lewis, P., et al.: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. NeurIPS (2020).
3. Robertson, S., Zaragoza, H.: The Probabilistic Relevance Framework: BM25 and Beyond. Foundations and Trends in Information Retrieval 3(4), 333–389 (2009).
4. Manning, C.D., Raghavan, P., Schütze, H.: Introduction to Information Retrieval. Cambridge University Press (2008).
5. Malkov, Y.A., Yashunin, D.A.: Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. IEEE Transactions on Pattern Analysis and Machine Intelligence 42(4), 824–836 (2020).
6. Cerebras Systems: Cerebras Cloud SDK Documentation (2024). <https://docs.cerebras.ai>
7. Chroma: Chroma — the AI-native open-source embedding database (2024). <https://docs.trychroma.com>
8. Meta AI: Llama 3: Open Foundation and Fine-Tuned Chat Models (2024). <https://ai.meta.com/llama/>